Formal Report on Hospital Management System in C

Title Page

Title: Formal Report on Hospital Management System in C

Authors:

- Archit Palshikar - 1262253366
- Bhuvee Jain - 1262254120
- Aqsa Khan - 1262253516

Generated by: Manus AI

Date: April 20, 2026

Acknowledgement

This report was generated by Manus AI, leveraging publicly available information and programming knowledge to provide a comprehensive overview of a Hospital Management System implemented in the C programming language.

Abstract

This report presents a detailed analysis of a Hospital Management System (HMS) developed using the C programming language. It covers the fundamental concepts, system architecture, design methodologies, and implementation details of such a system. The report aims to provide a clear understanding of how C can be utilized to create a functional and efficient HMS, capable of managing patient data, hospital information, and various administrative tasks. Key functionalities, data structures, and

algorithms are discussed, along with a literature review of existing systems and a conclusion summarizing the findings and potential future enhancements.

Table of Contents

1. Introduction

A Hospital Management System (HMS) is an integrated information system designed to manage all aspects of a hospital's operations, such as medical, administrative, financial, and legal issues, and the corresponding service processing. The primary goal of an HMS is to enhance the quality of patient care, improve operational efficiency, and reduce administrative costs. This report focuses on the development of a simple HMS using the C programming language, highlighting its core functionalities and structural components. C, known for its efficiency and low-level memory management capabilities, provides a robust foundation for building console-based applications that can effectively handle data management tasks within a hospital setting.

2. Literature Review

The concept of Hospital Management Systems has evolved significantly with advancements in computing technology. Early systems were often paper-based, leading to inefficiencies, data loss, and difficulties in information retrieval. The advent of computer programming languages like C enabled the development of automated systems that could store, process, and retrieve hospital data more effectively. Numerous projects and studies have explored the implementation of HMS using various programming languages. For instance, several console-based HMS projects in C focus on managing patient records, doctor appointments, and basic billing functionalities [1]. These systems often utilize structures to define data entities like `Hospital` and `Patient`, and functions for operations such as adding, viewing, searching, and sorting data [2]. The literature emphasizes the importance of a user-friendly interface, even in console applications, to ensure ease of use for hospital staff. While more complex, modern HMS often employ object-oriented languages and database management systems, C-based systems provide a foundational understanding of data management principles and direct control over system resources, making them valuable for educational purposes and simpler deployments.

3. System Design and Methodology

3.1 System Architecture

The proposed Hospital Management System in C follows a modular, console-based architecture. It is designed to be a standalone application, managing data primarily through in-memory structures or simple file I/O for persistence. The system is structured around key entities: `Hospital` and `Patient`. Each entity is represented by a C `struct` containing relevant attributes.

3.2 Data Structures

Hospital Structure:

```
struct Hospital {
    char name[50];
    char city[50];
    int beds;
    float price;
    float rating;
    int reviews;
};
```

Patient Structure:

```c
struct Patient {
    char name[50];
    int age;
};
```

3.3 Core Functionalities

The system incorporates the following core functionalities:

- **Data Entry:** Allowing the input of new hospital and patient records.
- **Data Display:** Presenting detailed information about hospitals and patients.
- **Data Sorting:** Organizing hospital data based on various criteria such as bed price, available beds, name, and rating/reviews.
- **Data Search:** Filtering hospitals by specific criteria, such as city.

3.4 Methodology

The development methodology for this C-based HMS involves a structured programming approach. Each functionality is implemented as a separate function, promoting code reusability and maintainability. The main function acts as a central control unit, presenting a menu-driven interface to the user and invoking appropriate functions based on user input. Data persistence can be achieved through basic file handling (e.g., saving data to text files), although for simplicity, initial implementations often focus on in-memory data management.

4. Implementation

The implementation of the Hospital Management System in C involves defining the data structures, writing functions for each operation, and integrating them through a `main` function with a menu-driven interface. Below are examples of key functions:

4.1 Header Files

Essential header files include `stdio.h` for input/output operations, `string.h` for string manipulation, and `ctype.h` for character type functions (e.g., `strcasecmp` for case-insensitive string comparison).

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

4.2 Function to Print Hospital Data

```c
void printHospital(struct Hospital hosp) {
    printf("Hospital Name: %s\n", hosp.name);
    printf("City: %s\n", hosp.city);
    printf("Total Beds: %d\n", hosp.beds);
    printf("Price per Bed: $%.2f\n", hosp.price);
    printf("Rating: %.1f\n", hosp.rating);
    printf("Reviews: %d\n", hosp.reviews);
    printf("\n");
}
```

4.3 Function to Sort Hospitals by Price

This function sorts hospitals based on the price per bed in ascending order using a bubble sort algorithm.

```c
void sortByPrice(struct Hospital hospitals[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (hospitals[j].price > hospitals[j + 1].price) {
                struct Hospital temp = hospitals[j];
                hospitals[j] = hospitals[j + 1];
                hospitals[j + 1] = temp;
            }
        }
    }
}
```

4.4 Main Function (Menu-Driven Interface)

The `main` function provides a loop that displays a menu of options to the user and executes the corresponding function based on their choice.

```c
int main() {
    // Sample hospital data initialization
    struct Hospital hospitals[5] = {
        {"Hospital A", "X", 100, 250.0, 4.5, 100},
        {"Hospital B", "Y", 150, 200.0, 4.2, 80},
        {"Hospital C", "X", 200, 180.0, 4.0, 120},
        {"Hospital D", "Z", 80, 300.0, 4.8, 90},
        {"Hospital E", "Y", 120, 220.0, 4.6, 110}
    };
    int n = 5; // Number of hospitals
    int choice;

    do {
        printf("\n\n\n*********** Hospital Management System 
Menu:***********\n\n");
        printf("1. Printing Hospital Data\n");
        printf("2. Printing Patients Data\n");
        printf("3. Sorting Hospitals by Beds Price\n");
        printf("4. Sorting Hospitals by Available Beds\n");
        printf("5. Sorting Hospitals by Name\n");
        printf("6. Sorting Hospitals by Rating and Reviews\n");
        printf("7. Print Hospitals in a Specific City\n");
        printf("8. Exit\n\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("\nPrinting Hospital Data:\n\n");
                for (int i = 0; i < n; i++) {
                    printHospital(hospitals[i]);
                }
                break;
            // ... other cases for sorting and searching
            case 8:
                printf("Exiting the program.\n");
                break;
            default:
                printf("Invalid choice. Please enter a valid option.\n");
        }
    } while (choice != 8);

    return 0;
}
```

5. Results and Discussion

The implementation of the Hospital Management System in C demonstrates the feasibility of managing basic hospital operations using a procedural programming paradigm. The system successfully allows for the input, display, sorting, and searching of hospital and patient data. The menu-driven interface provides a straightforward way for users to interact with the system. The use of `struct` for data representation and functions for modularity contributes to a clean and understandable codebase. However, as a console-based application, it lacks a graphical user interface (GUI) and relies on simple data storage mechanisms (e.g., in-memory arrays or basic file I/O), which can be limiting for large-scale deployments or complex data relationships. The sorting algorithms, such as bubble sort, are suitable for small datasets but would require optimization for larger volumes of data. Despite these limitations, the C-based HMS serves as an excellent educational tool for understanding fundamental programming concepts and system design principles.

6. Conclusion

This report has outlined the design, methodology, and implementation of a Hospital Management System using the C programming language. The system effectively demonstrates core functionalities required for managing hospital and patient information. While simple in its current form, it provides a solid foundation for further development, including the integration of more advanced features like persistent data storage (using files or databases), a graphical user interface, and more sophisticated data management algorithms. The project underscores the power and flexibility of C in developing efficient and robust applications for various domains, including healthcare management.

7. References

[1] GeeksforGeeks. (2023, September 25). *Hospital Management System in C*. Retrieved from https://www.geeksforgeeks.org/c/c-program-for-hospital-management-system/
[2] Aticleworld. (2022, December 23). *Hospital Management System Project in C*. Retrieved from https://aticleworld.com/hospital-management-system-project-in-c/